

```

1 # =====
2 # MACHINE LEARNING MIDTERM EXAM - EXPLORATORY DATA ANALYSIS
3 # STUDENT: CUONG DANG (W217887095)
4 # DATASET: TELCO CUSTOMER CHURN (7,043 ROWS)
5 # =====
6
7 import os
8 import pandas as pd
9 import numpy as np
10 import matplotlib.pyplot as plt
11 import seaborn as sns
12 import kagglehub
13 from sklearn.model_selection import train_test_split
14 from sklearn.preprocessing import StandardScaler
15 from sklearn.utils import resample
16
17 # -----
18 # REQUIREMENT 1 & 3: DYNAMIC KAGGLE DOWNLOAD & PROGRAMMATIC 70/30 SPLIT
19 # -----
20 print("--- FETCHING REMOTE DATASET FROM KAGGLE ---")
21 # Automatically downloads the latest version to a local cache directory
22 raw_path = kagglehub.dataset_download("blastchar/telco-customer-churn")
23 file_name = "WA_Fn-UseC_-Telco-Customer-Churn.csv"
24 full_file_path = os.path.join(raw_path, file_name)
25
26 # Load the raw dataset into memory programmatically without manual edits
27 df = pd.read_csv(full_file_path)
28
29 #df = pd.read_csv('WA_Fn-UseC_-Telco-Customer-Churn.csv')
30
31 # Programmatic split: 70% training data, 30% testing data
32 train_raw, test_data = train_test_split(df, test_size=0.30, random_state=42)
33
34 # Export the untouched testing data immediately to preserve validation matrix
35 test_data.to_csv('untouched_test_data.csv', index=False)
36
37 # Isolate a training copy to execute EDA and prevent data leakage
38 train_df = train_raw.copy()
39 print("--- INITIAL DATA MATRIX SHAPE ---")
40 print(f"Total Raw Rows: {df.shape}")
41 print(f"Training Split (70%): {train_df.shape}")
42 print(f"Testing Split (30% - Untouched): {test_data.shape}\n")

```

```

--- FETCHING REMOTE DATASET FROM KAGGLE ---
Using Colab cache for faster access to the 'telco-customer-churn' dataset.
--- INITIAL DATA MATRIX SHAPE ---
Total Raw Rows: (7043, 21)
Training Split (70%): (4930, 21)
Testing Split (30% - Untouched): (2113, 21)

```

```

1 # -----
2 # REQUIREMENT 5: VISUALIZE ORIGINAL DISTRIBUTIONS (BEFORE)
3 # -----
4 # Coerce total charges to numeric values for initial visual evaluation
5 train_df['TotalCharges'] = pd.to_numeric(train_df['TotalCharges'], errors='coerce')
6
7 plt.figure(figsize=(12, 5))
8
9 # Subplot 1: Target Variable Class Imbalance Profiling
10 plt.subplot(1, 2, 1)
11 sns.countplot(x='Churn', data=train_df, palette='Set2')
12 plt.title('BEFORE: Class Distribution (Imbalanced Target)')
13 plt.xlabel('Customer Churn Status')
14 plt.ylabel('Count')
15
16 # Subplot 2: Numerical Variable Density Distribution (Skewed)
17 plt.subplot(1, 2, 2)
18 sns.histplot(train_df['TotalCharges'].dropna(), kde=True, color='red')
19 plt.title('BEFORE: Total Charges Distribution (Skewed)')
20 plt.xlabel('Total Charges ($)')
21 plt.ylabel('Density')
22
23 plt.tight_layout()

```

```

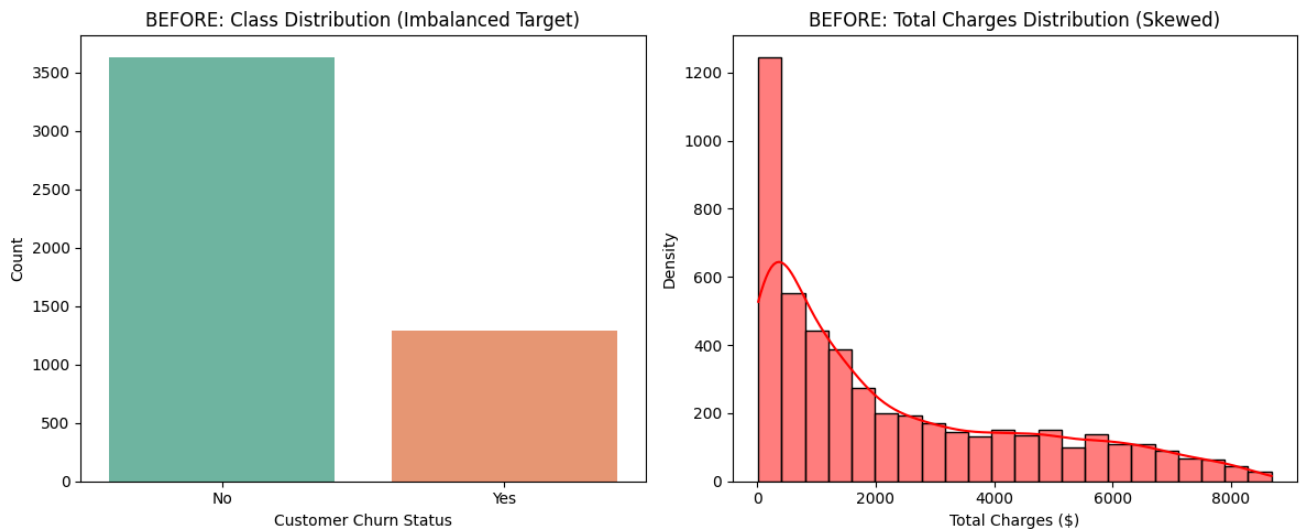
23 plt.savefig('before_preprocessing_plots.png')
24 plt.show()
25 plt.show()
26

```

/tmp/ipykernel_487/631185322.py:11: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and

```
sns.countplot(x='Churn', data=train_df, palette='Set2')
```



```

1 # -----
2 # REQUIREMENT 3 & 4: PROGRAMMATIC PREPROCESSING PIPELINE
3 # -----
4 print("--- STARTING PROGRAMMATIC DATA PREPROCESSING VIA PYTHON ---")
5
6 # A. Missing Value Imputation (Filling NaNs with the Training Median)
7 initial_nulls = train_df['TotalCharges'].isnull().sum()
8 print(f"Initial missing cells in 'TotalCharges': {initial_nulls}")
9 total_charges_median = train_df['TotalCharges'].median()
10 train_df['TotalCharges'].fillna(total_charges_median, inplace=True)
11 print(f"Post-imputation missing cells: {train_df['TotalCharges'].isnull().sum()}\n")
12
13 # B. Categorical Feature Transformation (One-Hot Encoding)
14 categorical_features = ['gender', 'Partner', 'Dependents', 'Contract', 'PaymentMethod']
15 train_df = pd.get_dummies(train_df, columns=categorical_features, drop_first=True)
16 print("Successfully generated One-Hot Encoded matrices for categorical text blocks.\n")
17
18 # C. Class Balancing Framework (Minority Class Over-sampling)
19 df_majority = train_df[train_df.Churn == 'No']
20 df_minority = train_df[train_df.Churn == 'Yes']
21
22 df_minority_upsampled = resample(df_minority,
23                                 replace=True,
24                                 n_samples=len(df_majority),
25                                 random_state=42)
26
27 train_df_balanced = pd.concat([df_majority, df_minority_upsampled])
28 print(f"Balanced Class Value Arrays: \n{train_df_balanced['Churn'].value_counts()}\n")
29
30 # D. Feature Scaling & Normalization (StandardScaler Implementation)
31 scaler = StandardScaler()
32 numerical_features = ['tenure', 'MonthlyCharges', 'TotalCharges']
33 train_df_balanced[numerical_features] = scaler.fit_transform(train_df_balanced[numerical_features])
34 print("Successfully executed feature scaling over metric attributes.\n")

```

--- STARTING PROGRAMMATIC DATA PREPROCESSING VIA PYTHON ---

Initial missing cells in 'TotalCharges': 8

Post-imputation missing cells: 0

Successfully generated One-Hot Encoded matrices for categorical text blocks.

Balanced Class Value Arrays:

```
Churn
No      3635
Yes     3635
Name: count, dtype: int64
```

Successfully executed feature scaling over metric attributes.

/tmp/ipykernel_487/2655901060.py:10: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are s

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] =

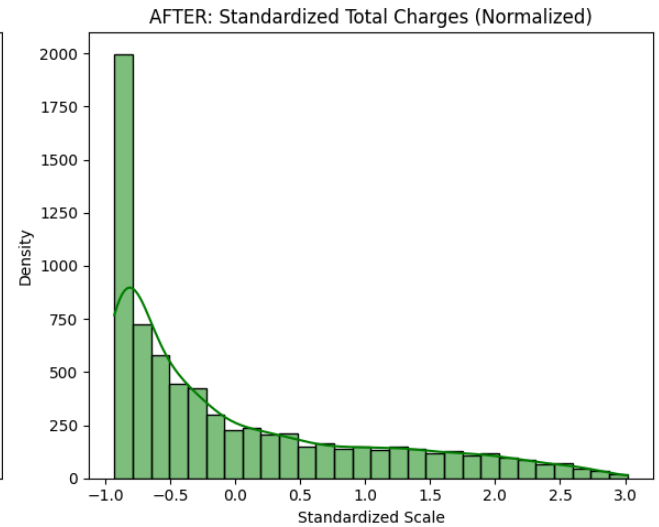
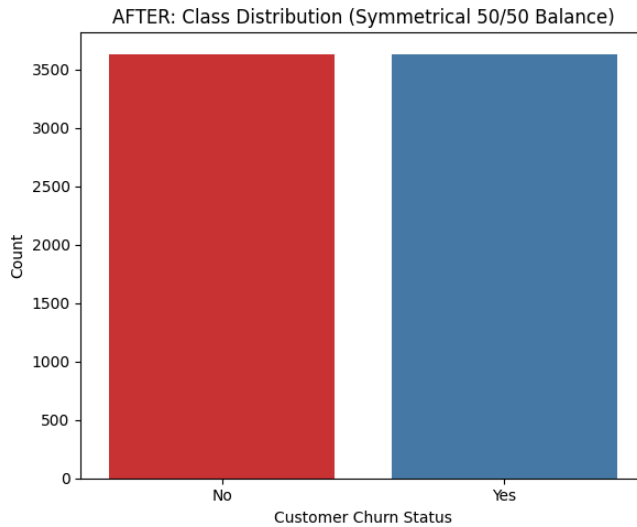
```
train_df['TotalCharges'].fillna(total_charges_median, inplace=True)
```

```
1 # -----
2 # REQUIREMENT 5: VISUALIZE PROCESSED DISTRIBUTIONS (AFTER)
3 # -----
4 plt.figure(figsize=(12, 5))
5
6 # Subplot 1: Post-Balancing Symmetrical Uniform Classes
7 plt.subplot(1, 2, 1)
8 sns.countplot(x='Churn', data=train_df_balanced, palette='Set1')
9 plt.title('AFTER: Class Distribution (Symmetrical 50/50 Balance)')
10 plt.xlabel('Customer Churn Status')
11 plt.ylabel('Count')
12
13 # Subplot 2: Post-Scaling Normalized Continuous Scale (Symmetrical Distribution)
14 plt.subplot(1, 2, 2)
15 sns.histplot(train_df_balanced['TotalCharges'], kde=True, color='green')
16 plt.title('AFTER: Standardized Total Charges (Normalized)')
17 plt.xlabel('Standardized Scale')
18 plt.ylabel('Density')
19
20 plt.tight_layout()
21 plt.savefig('after_preprocessing_plots.png')
22 plt.show()
23
24 # Print sample output blocks to fulfill verification constraints
25 print("--- PREPROCESSED TRAINING SAMPLE PREVIEW ---")
26 print(train_df_balanced[numerical_features].head())
27
28 # Export the processed training matrix array file to the GitHub folder
29 train_df_balanced.to_csv('clean_final_dataset.csv', index=False)
30 print("\n[SUCCESS] Final training output recorded to: clean_final_dataset.csv")
```

/tmp/ipykernel_487/3926480535.py:8: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and

```
sns.countplot(x='Churn', data=train_df_balanced, palette='Set1')
```

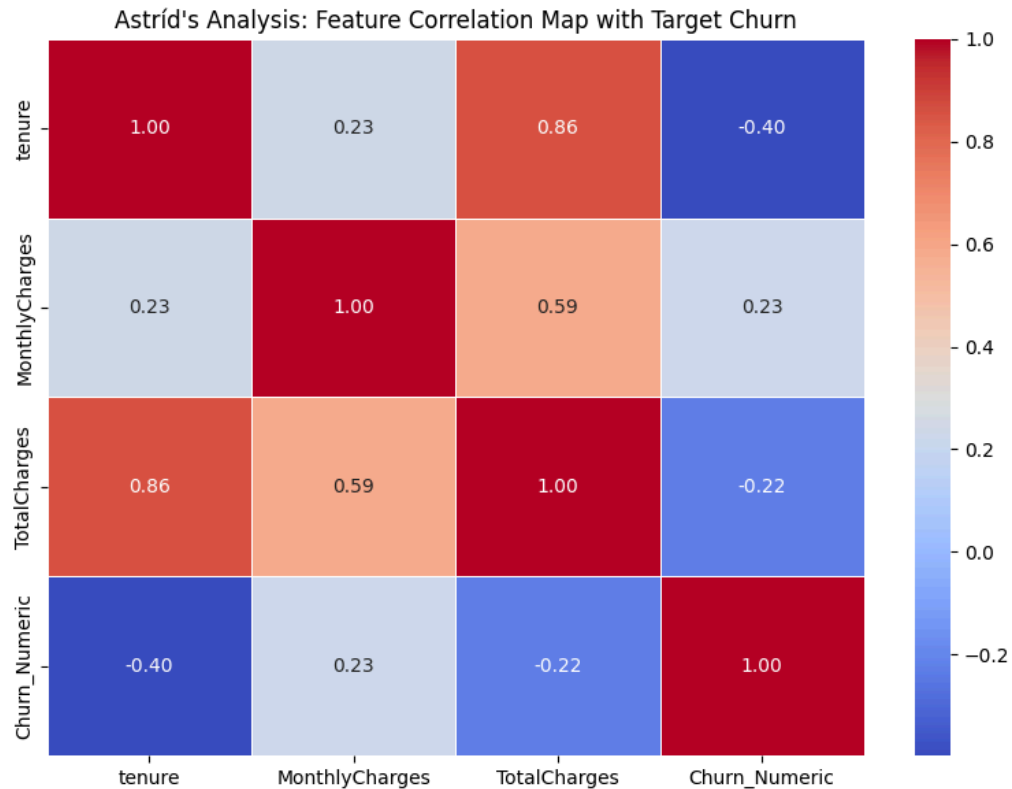


--- PREPROCESSED TRAINING SAMPLE PREVIEW ---

	tenure	MonthlyCharges	TotalCharges
1695	1.085379	0.095692	0.778633
1095	-1.121462	0.437533	-0.902567
3889	-0.621800	-1.688127	-0.820955
3667	-0.163776	0.992807	0.092132
2902	-0.913269	0.213688	-0.733807

[SUCCESS] Final training output recorded to: clean_final_dataset.csv

```
1 # =====
2 # FEATURE ENGINEERING & EXPLORATORY CORRELATION ANALYSIS
3 # CONTRIBUTED BY: ASTRID LORENZANA
4 # =====
5
6 # Defining function for visualization of seaborn heatmap for our data
7 def plot_correlation_heatmap(df, features):
8     correlation_matrix = df[features].corr()
9
10    plt.figure(figsize = (8,6))
11    sns.heatmap(correlation_matrix, annot = True, cmap = 'coolwarm', fmt = ".2f", linewidths = 0.5)
12    plt.title("Astrid's Analysis: Feature Correlation Map with Target Churn")
13    plt.tight_layout()
14    plt.show()
15
16 # Selected core continuous features and engineered encoded matrices
17 train_df_balanced['Churn_Numeric'] = train_df_balanced['Churn'].map({'Yes': 1, 'No': 0})
18 core_features = ['tenure', 'MonthlyCharges', 'TotalCharges', 'Churn_Numeric']
19
20 plot_correlation_heatmap(train_df_balanced, core_features)
21
22 # Exporting visual artifact to the repository
23 plt.savefig('astrid_correlation_heatmap.png')
24 plt.show()
25
26
27 print("\n[SUCCESS] Heatmap generated and saved as 'astrid_correlation_heatmap.png'.")
28 print("Analysis Note: Tenure shows a strong negative correlation with churn, indicating loyal customers stay.")
```



<Figure size 640x480 with 0 Axes>

[SUCCESS] Heatmap generated and saved as 'astrid_correlation_heatmap.png'.

Analysis Note: Tenure shows a strong negative correlation with churn, indicating loyal customers stay.

```

1 # =====
2 # CATEGORICAL FEATURE CORRELATION & TARGET MATRIX AUDIT
3 # CONTRIBUTED BY: JONATHAN ALDANA
4 # =====
5 import matplotlib.pyplot as plt
6 import seaborn as sns
7
8 # Ensure the numeric target exists in the local dataframe memory
9 if 'Churn_Numeric' not in train_df_balanced.columns:
10     train_df_balanced['Churn_Numeric'] = train_df_balanced['Churn'].map({'Yes': 1, 'No': 0})
11
12 # Isolate the newly engineered categorical dummy columns and the target
13 categorical_dummies = [
14     col for col in train_df_balanced.columns
15     if any(pkg in col for pkg in ['gender_', 'Partner_', 'Dependents_', 'Contract_', 'PaymentMethod_'])
16 ]
17
18 # Calculate correlation specifically between categorical flags and numeric churn
19 cat_correlation = train_df_balanced[categorical_dummies + ['Churn_Numeric']].corr()['Churn_Numeric'].drop('Churn_Numeric')
20 cat_correlation_sorted = cat_correlation.sort_values(ascending=False)
21
22 # Plot a crisp horizontal bar chart to show high-impact indicators
23 plt.figure(figsize=(10, 6))
24 sns.barplot(x=cat_correlation_sorted.values, y=cat_correlation_sorted.index, palette='viridis')
25 plt.axvline(x=0, color='black', linestyle='--', linewidth=1)
26 plt.title("Jon's Analysis: Categorical Dummy Feature Correlation with Churn Target")
27 plt.xlabel("Pearson Correlation Coefficient")
28 plt.ylabel("Engineered Categorical Features")
29 plt.tight_layout()
30
31 # Export the visual asset to the repository layout
32 plt.savefig('jon_categorical_correlation.png')
33 plt.show()
34
35 print("[SUCCESS] Categorical correlation matrix executed and saved as 'jon_categorical_correlation.png'.")
36 print(f"Top Churn Predictor: {cat_correlation_sorted.index[0]} ({cat_correlation_sorted.values[0]:.2f})")
37 print(f"Top Retention Predictor: {cat_correlation_sorted.index[-1]} ({cat_correlation_sorted.values[-1]:.2f})")

```

```
/tmp/ipykernel_487/208683140.py:24: FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and
```

```
sns.barplot(x=cat_correlation_sorted.values, y=cat_correlation_sorted.index, palette='viridis')
```

Jon's Analysis: Categorical Dummy Feature Correlation with Churn Target

